

MÓDULO 6

TESTES

☐ Testes de aceitação

Devem ser realizados para capacitar o usuário a validar todos os requisitos do software

•Este passo do teste da validação irá desprender um esforço de tempo que irá depender do tamanho e complexidade do software e da necessidade de realizar sua demonstração (podendo ser feito através de uma passagem rápida sobre os principais pontos ou realizando a uma série de testes planejados), sempre com o objetivo de descobrindo erros cumulativos que levariam a destruição do software ao longo do tempo.

•Alternativas para o teste de aceitação são: teste alfa e teste beta.

•

–No teste alfa, o software é usado num ambiente natural e controlado, onde o desenvolvedor instala o software e observa os erros e problemas decorridos com o usuário.

–No teste beta as instalações são feitas pelo usuário final e sem a presença do desenvolvedor, sendo assim uma aplicação real do software em um ambiente não controlado, onde o cliente registra todos os problemas que são encontrados durante o teste e os relata posteriormente ao desenvolvedor.

☐ Teste de Recuperação

•Os softwares precisam recuperar-se de erros e retornar o processamento dentro de um tempo previamente especificado, evitando que a função global do sistema cesse e conseqüentemente que prejuízos econômicos ocorram.

•Definição: É um teste que força o software a falhar de diversos modos, e verifica se a recuperação é adequadamente realizada dentro de um limite de tempo aceitável e se os dados são avaliados corretamente.

☐ Teste de Segurança

•Definição: Teste que verifica se todos os mecanismos de proteção do sistema, protegerão o software de invasão imprópria ou ilegal. Durante o teste de segurança, o analista desempenha o papel da pessoa que deseja invadir o sistema.

•Exemplos:

- Tentar obter senhas de funcionários externos;*
- Projetar uma ferramenta para derrubar o sistema;*
- Provocar erros no sistema de propósito para invadí-lo durante a recuperação;*

•Objetivo: Tentar dificultar ao máximo o acesso do indivíduo e fazer com que a invasão não valha a pena.

□Teste de Estresse

•Definição: Teste que executa o software de forma a exigir recursos em quantidade, frequência ou volume anormais.

•Para isso devem ser projetados casos de teste que:

- exijam o máximo da memória;*
- gerem um número maior de interrupções que o número projetado;*
- realizem diversas buscas de dados e verifiquem como as funções de entrada responderão, com muitas entradas de dados. Ou seja, qualquer caso de teste que tente, essencialmente, derrubar o software.*

□Teste de Desempenho

•O último teste a ser realizado é o teste de desempenho.

•Este teste deve ser acoplado ao teste de estresse, onde será avaliado o desempenho do software, medindo a utilização de recurso, monitorando intervalos de execução, registrando eventos ocorridos e retirando amostras do comportamento da máquina.

❖Quais são os objetivos dos Testes?

- Teste é um processo de execução de um programa com a finalidade de encontrar um erro.*
- Um bom caso de teste é aquele que tem alta probabilidade de encontrar um erro ainda não descoberto.*
- Um teste bem-sucedido é aquele que descobre um erro ainda não descoberto.*

❖Os Princípios dos Testes

Foram elencados 5 princípios mais importantes de testes os quais são descritos a seguir:

- 1.Todos os testes devem estar relacionados aos requisitos dos clientes*
- 2.Os testes devem ser planejados muito antes de serem iniciados*
- 3.O princípio de Pareto se aplica aos testes de software*
- 4.Os testes devem começar "no varejo" e progredir até o teste "no atacado"*
- 5.Testes exaustivos não são possíveis*

❖O que é Testabilidade ?

A testabilidade de um software é simplesmente o quão fácil um programa de computador pode ser testado.

❖ O que determina a testabilidade do software?

As seguintes características indicam a testabilidade de um software:

- Operabilidade : Quanto melhor funciona mais eficiente pode ser testado;
- Observabilidade: O que você vê é o que você testa
- Controlabilidade: Quanto melhor você pode controlar o software, mais o teste pode ser automatizado.
- Decomponibilidade: Capacidade de isolar problemas. – Construção em módulos
- Simplicidade : Quanto menos houver para testar, mais rápido o mesmo será testado.
- Estabilidade: Quanto menos modificações, menos interrupções no teste.
- Compreensibilidade: Quanto mais informações temos, mais racionalmente vamos testar.

❖Tipos de Testes

□ Caixa-Branca

Os testes de caixa-branca também são conhecidos como testes de caixa de vidro, ele é um teste baseado na estrutura do programa, nos controles internos do código fonte do sistema desenvolvido. Esse teste tende a passar por todos os caminhos lógicos do sistema.

□ Objetivos

- *Garantir que os caminhos independentes de um módulo tenham sido exercitados pelo menos uma vez*
- *Exercitem todas as decisões lógicas em seus lados verdadeiros e falsos*
- *Executem todos os ciclos nos seus limites e dentro de seus intervalos operacionais*

Exercitem as estruturas de dados internas para garantir sua validade.

❖ **Exemplo de teste Caixa – Branca**

- Teste de caminho básico
 - Notação de Grafos e Fluxos
 - Caminhos independentes de programas
 - Matrizes de Grafos
- Teste de Estrutura de Controle
 - Teste de Condição
 - Teste de Fluxo de Dados
 - Teste de ciclo

❑ **Caixa-Preta**

Um teste de caixa-preta também é conhecido como teste comportamental, focaliza os requisitos funcionais do software. Isto é, o teste de caixa-preta permite derivar um conjunto de condições de entrada que vão exercitar plenamente todos os requisitos funcionais do sistema.

❑ **Erros que o teste caixa-preta encontra:**

- *Funções incorretas ou omitidas*
- *Erros de interface*
- *Erros de estrutura de dados ou acessos a bases externas*
- *Erros de comportamento ou desempenho*
- *Erros de iniciação ou término*

✓ *Diferente dos testes de caixa-branca que é realizado no início do processo de teste, esses testes podem ser realizados durante os últimos estágios de teste.*

❖ **Exemplo de Teste Caixa-Preta**

- Métodos de testes baseados em Grafos

- Particionamento de Equivalência
- Análise de Valor-limite
- Teste de Matriz Ortogonal

❑ **Teste de unidade**

•*O teste de unidade concentra-se na verificação da menor unidade de projeto do software. Usando a descrição de projeto para testar os caminhos de controle do código-fonte e descobrir erros nos limites das unidades*

•*Esta fase é realizada para garantir que: a interface controla a entrada e a saída de dados adequadamente, a unidade funciona corretamente nos valores limites, caminhos de manipulação de erro são executados, comandos das unidades tenham sido exercitados pelo menos uma vez.*

•*Geralmente um componente não funciona isoladamente, demandando o desenvolvimento de outros softwares denominados pseudo-controladores e pseudo-controlados para realizar cada teste de unidade, agregando despesas indiretas ao software.*

•*A equipe de desenvolvimento do software é sempre responsável por testar as unidades individuais do software, para garantir que cada uma execute a função para a qual foi projetada. Em muitos casos, a equipe de desenvolvimento também realiza testes de integração.*

❑ **Teste de Integração**

•*Mesmo realizando o teste de unidade, o software pode causar erros ao integrar suas unidade*

•*Problemas:*

–*Dados perdidos*

–*Unidades podem provocar erros em outras.*

–*Subfunções quando combinadas podem não produzir o efeito desejado.*

–*Imprecisão aceitável em níveis de unidades pode ser ampliadas para níveis não aceitáveis.*

–*Estruturas de dados globais podem apresentar problemas.*

•*É uma técnica sistemática para construir a estrutura do software, realizando ao mesmo tempo testes para descobrir erros associados às interfaces.*

- Abordagens big-bang:**

- Tendência dos programadores.*

- Problemas:*

- Difícil correção dos erros, devido ao vasto espaço do programa.*

- Solução:*

- o software deve ser integrado de forma incremental, ou seja, o software é construído e testado em pequenos segmentos, onde as interfaces podem ser testadas completamente e os erros são mais fáceis de serem encontrados e corrigidos.*

□Teste de regressão

- O teste de regressão é a atividade que ajuda a garantir que modificações(devido ao teste ou a outras razões) não introduzem comportamento indesejável ou erros adicionais.

□Teste de Validação

- A validação será bem sucedida somente quando o software funcionar de maneira esperada pelo cliente.

- A validação é realizada através de uma série de testes que demonstram que os requisitos foram atendidos.

- Um plano de teste descreve as classes de teste a serem conduzidas e um procedimento de teste define os casos de teste que serão usados para demonstrar a validação dos requisitos.

- Tanto o plano quanto o procedimento são criados para garantir que os requisitos são satisfeitos

- Depois de ser realizado cada um dos casos de teste de validação, pelo menos uma das duas condições deve ser atendida:

- As características de função ou desempenho satisfazem as especificações e são aceitas;

- Um desvio ou erro de especificação gera um item de uma lista de correções a serem realizadas.

